

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Laboratorio de control automático

Profesor: Pablo Cesar Rodríguez

Diseño y Construcción de un Sistema de Levitación Neumática

Jeremy Soto Chacón

jeremysotoch@estudiantec.cr

Vladimir González Morera

vlachogonzalez@estudiantec.cr

Pablo Navarro Robles

pablonavarobles0102@estudiantec.cr

ANTIGRAVITY

II Semestre 2025

Índice

1. Introducción	4
1.1. Contextualización	4
1.2. Presentación del Sistema	5
1.3. Objetivos de Control	5
1.3.1. Mantener Altura Constante	5
1.3.2. Seguimiento de Trayectoria	5
1.3.3. Minimizar Oscilaciones	5
2. Proceso de Construcción	5
2.1. Diseño Conceptual	6
2.1.1. Descripción general del sistema	6
2.1.2. Especificaciones de Diseño	7
2.2. Materiales y Componentes	7
2.2.1. Componentes estructurales	7
2.2.2. Componentes electrónicos	8
2.3. Diseño Estructural	9
2.4. Diseño Electrónico	10
2.4.1. Esquemático General	10
2.4.2. Regulación y Distribución de Potencia ($12\text{ V} \rightarrow 5\text{ V}$)	11
2.4.3. Actuación y Control PWM del Ventilador	12
2.4.4. Sensado y Usuario	13
2.4.5. Asignación de Señales	13
3. Obtención del Modelo Empírico	14
3.1. Datos experimentales	14
3.2. Identificación con <i>System Identification Toolbox</i>	14
3.3. Modelo discreto equivalente	15
4. Obtención del Controlador de la Planta	15
4.1. Criterios de diseño	16
4.2. Sintonización con <i>PID Tuner</i>	16
4.3. Ajustes prácticos en la planta real	17
4.4. Controlador resultante	17
5. Controlador Digital	18
5.1. Variables y estructura del PID incremental	19
5.2. Modo automático: ganancias fijas	19
5.3. Modo manual: ajuste experimental de K_p , K_i y K_d	20
6. Resultados	21
6.1. Pruebas de seguimiento de referencia en modo automático	21
6.2. Funcionamiento en modo manual	22

7. Análisis de Resultados	22
7.1. Modelo lineal vs. planta real	22
7.2. Impacto del escalado de las ganancias	23
7.3. Efectos de la implementación digital	23
7.4. Utilidad del modo manual	24
8. Conclusiones	24
9. Apéndices	25
A. Video de funcionamiento del sistema	25
B. Imágenes de componentes estructurales	26
C. Imágenes de componentes electrónicos	36
D. Planos de la planta	46
E. Codigó de la planta	50

Resumen

Este documento presenta el diseño, construcción e implementación de un sistema de levitación neumática para el control de posición de una esfera ligera dentro de un tubo vertical. Se describe el desarrollo de la estructura mecánica (columnas en PVC, tubo y lámina acrílica, piezas impresas en PLA) y la integración electrónica basada en un microcontrolador Arduino Uno, un sensor óptico TF-Luna para la medición de altura, un ventilador de 12 V como actuador principal y una etapa de potencia con L293D y regulación lineal $12\text{ V} \rightarrow 5\text{ V}$ mediante 7805.

A partir de datos experimentales en lazo abierto, obtenidos aplicando un escalón de PWM al ventilador, se identificó un modelo empírico de la planta usando *System Identification Toolbox*, seleccionando una función de transferencia continua de tercer orden con un cero y un ajuste del 87.25 %. Este modelo se discretizó con un periodo de muestreo $T_s = 0,01\text{ s}$ y se utilizó como base para el diseño de un controlador PID mediante *PID Tuner*. Posteriormente, las ganancias se ajustaron (escalado de K_p y K_i) para compensar la zona muerta del ventilador y garantizar el arranque de la levitación en la planta real.

El controlador PID se implementó en forma incremental en el Arduino, con modos de operación automático (ganancias fijas) y manual (sintonía en línea mediante potenciómetros). Los resultados experimentales muestran que el sistema logra levitar la esfera y seguir referencias de altura con estabilidad, aunque con sobreimpulsos mayores a los predichos por el modelo lineal debido a las no linealidades del actuador y a las limitaciones de la implementación digital. El sistema resultante constituye una plataforma didáctica para el estudio práctico de identificación de sistemas, diseño de controladores PID e implementación embebida en tiempo real. Adicionalmente, se registró un video demostrativo del funcionamiento del sistema, disponible en el Apéndice A.

Palabras clave— levitación neumática; controlador PID digital; Arduino Uno; TF-Luna; modulación PWM; ventilador 12 V; L293D; regulador 7805; identificación de sistemas; System Identification Toolbox; PID Tuner; modelo empírico; control en lazo cerrado; implementación embebida; zona muerta de actuadores.

1. Introducción

1.1. Contextualización

Los sistemas de levitación neumática representan una aplicación práctica de los principios fundamentales de la teoría de control automático, combinando dinámica de fluidos, sensores y sistemas embebidos como Arduino o ESP32 entre otros. Estos sistemas, caracterizados por su naturaleza inestable y no lineal, constituyen un desafío interesante en el ámbito del control, ya que requieren una respuesta rápida y precisa para mantener un objeto suspendido en el aire mediante un flujo controlado de aire.

La levitación neumática tiene aplicaciones en diversos campos industriales y científicos, incluidos sistemas de transporte sin contacto, mesas de flotación para procesos de manufactura sensibles, simuladores de microgravedad y equipos de investigación en dinámica de fluidos. En el contexto académico, estos sistemas sirven como plataformas ideales para el estudio y validación experimental de técnicas de control moderno, permitiendo observar de

manera tangible conceptos como estabilidad, respuesta transitoria, rechazo de perturbaciones y sintonización de controladores. [1]

1.2. Presentación del Sistema

El presente informe documenta el diseño, construcción e implementación de un sistema de levitación neumática vertical para el control de posición de una esfera. El sistema consiste en una estructura vertical que confina un flujo de aire ascendente generado por un ventilador de 12V, dentro del cual se suspende una esfera ligera de estereofón. La posición vertical de la esfera es medida mediante un sensor laser TF-Luna, que proporciona retroalimentación continua al sistema de control.

El sistema implementa un controlador PID (Proporcional-Integral-Derivativo) que ajusta dinámicamente la velocidad del ventilador mediante modulación por ancho de pulso (PWM), permitiendo regular el flujo de aire y, consecuentemente, la fuerza de sustentación sobre la esfera. La arquitectura del sistema integra sensores de precisión, actuadores mediante control PWM y procesamiento digital en tiempo real con un microcontrolador; todo esto con el fin de lograr un comportamiento estable del sistema en lazo cerrado.

1.3. Objetivos de Control

1.3.1. Mantener Altura Constante

Diseñar e implementar un controlador capaz de mantener la esfera en una posición vertical de referencia, compensando perturbaciones externas y variaciones en las condiciones del sistema. Se busca minimizar el error en estado estacionario y lograr que el sistema sea robusto ante cambios en la carga o perturbaciones en el flujo de aire.

1.3.2. Seguimiento de Trayectoria

Desarrollar la capacidad del sistema para seguir referencias variables en el tiempo, permitiendo que la esfera se desplace de manera controlada a diferentes alturas dentro del rango operativo del sistema. Esto implica lograr un tiempo de estabilización adecuado y una respuesta dinámica que minimice el sobreimpulso durante las transiciones.

1.3.3. Minimizar Oscilaciones

Sintonizar los parámetros del controlador PID para reducir al mínimo las oscilaciones no deseadas alrededor del punto de referencia, mejorando la estabilidad del sistema y la suavidad de la respuesta. Este objetivo busca un comportamiento amortiguado que garantice la calidad del control y la eficiencia energética del sistema.

2. Proceso de Construcción

En esta sección se documenta el proceso completo de construcción del sistema de levitación neumática, desde el diseño conceptual hasta el ensamblaje final. Se describen los

materiales empleados, los componentes electrónicos integrados y el procedimiento sistemático seguido para la fabricación de cada subsistema. Adicionalmente, se presentan los planos de construcción que permiten la replicabilidad del diseño.

2.1. Diseño Conceptual

El diseño conceptual establece los lineamientos generales del sistema y define las especificaciones técnicas que guiarán la construcción. El sistema de levitación neumática se concibe como una estructura vertical compacta que integra todos los componentes necesarios para el control de posición de una esfera mediante flujo de aire ascendente.

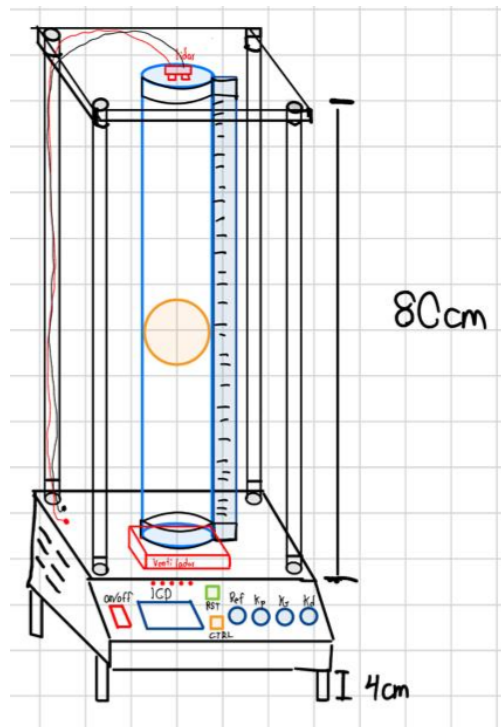


Figura 1: Boceto del diseño

2.1.1. Descripción general del sistema

El sistema consiste en una estructura de 80 cm de altura total, con una base que aloja los componentes electrónicos y el sistema de actuación. La configuración vertical permite un rango operativo amplio para el desplazamiento de la esfera, maximizando las posibilidades de control y seguimiento de trayectoria. Los elementos principales del diseño incluyen:

Estructura Vertical: Compuesta por cuatro columnas que proporcionan soporte mecánico y estabilidad al conjunto. Estas columnas delimitan el espacio de trabajo y sirven como guías visuales para el rango de operación del sistema.

Tubo de Levitación: Elemento cilíndrico transparente ubicado en el centro de la estructura, que confina el flujo de aire y permite la visualización del comportamiento de la esfera. El tubo cuenta con una escala graduada lateral que facilita la medición visual de la posición de la esfera y la verificación del desempeño del sistema de control.

Sensor Infrarrojo: Posicionado en la parte superior del tubo, el sensor TF-Luna mide continuamente la distancia hacia la esfera, proporcionando la retroalimentación necesaria para el lazo de control. Su ubicación superior permite una lectura directa de la altura de la esfera con respecto a la base del sistema.

Sistema de Actuación: El ventilador de 12V se encuentra montado en la base inferior del tubo de levitación, generando un flujo de aire ascendente controlado mediante señal PWM. La geometría de acoplamiento entre el ventilador y el tubo está diseñada para optimizar la eficiencia del flujo y minimizar las turbulencias.

Base de Control: Plataforma inferior que integra la electrónica de control, incluyendo el microcontrolador, el driver o puente H del motor, la fuente de alimentación y elementos de interfaz de usuario (pantalla LCD, potenciómetros y botones). Esta disposición centraliza todos los componentes electrónicos, facilitando el acceso para configuración y mantenimiento.

2.1.2. Especificaciones de Diseño

Las especificaciones técnicas del sistema son las siguientes:

- Altura total de la estructura: 85 cm (aproximadamente)
- Altura de la base: 4 cm
- Rango operativo de levitación: 0-80cm (aproximadamente)
- Diámetro del tubo de levitación:
- Tensión de alimentación del ventilador: 12V DC
- Sensor de posición: TF-Luna (rango 0-8m, precisión ± 1 cm)
- Interfaz de usuario: Pantalla LCD, 4 potenciómetros de ajuste (Kp, Ki, Kd, Ref), botones de control y switches

2.2. Materiales y Componentes

A continuación se presenta la lista completa de materiales estructurales y componentes electrónicos utilizados en la implementación del sistema de levitación neumática. La selección de cada elemento se realizó considerando las especificaciones de diseño, disponibilidad y compatibilidad entre componentes.

2.2.1. Componentes estructurales

- **Estructura (columnas y soportes)**
 - 4 tubos de PVC 1/2" de 85 cm de longitud (véase Fig. 10).
 - 12 conectores de PVC 1/2" (véase Fig. 11).
 - 4 tapones de PVC 1/2" (véase Fig. 12).
- **Plataforma superior y escala**

- 1 lámina acrílica transparente, 3 mm, 100 cm × 100 cm (corte láser) (véase Fig. 13).

■ **Tubo de levitación**

- Tubo acrílico transparente, $L = 85$ cm, $\varnothing$ interior [X] mm, $\varnothing$ exterior [Y] mm (véase Fig. 14).

■ **Piezas impresas y base**

- 1 rollo de PLA negro, $\varnothing$ 1.75 mm, 1 kg (véase Fig. 15).

■ **Fijaciones y herrajes**

- 50 tornillos, 50 tuercas y 50 arandelas (métrica según diseño) (véase Fig. 16).

■ **Elemento levitado**

- 1 bola de estereofón (EPS) de $\varnothing$ [X] mm (véase Fig. 17).

2.2.2. Componentes electrónicos

■ **Control y lógica**

- 1 Arduino Uno (véase Fig. 30).
- 1 pantalla LCD 20×4 (idealmente con adaptador I²C) (véase Fig. 31).
- 4 potenciómetros 10k Ω (véase Fig. 32).
- 1 pulsador (véase Fig. 33).
- 1 switch general (véase Fig. 34).
- 1 switch de 3 estados (véase Fig. 35).
- 20 conectores (pins) macho para Arduino (véase Fig. 36).
- 20 cables jumper hembra-macho (véase Fig. 37).

■ **Sensado**

- 1 sensor TF-Luna (ToF; UART/I²C) (véase Fig. 38).

■ **Actuación y potencia**

- 1 ventilador 12 V (véase Fig. 39).
- 1 puente H L293D (véase Fig. 40).
- 2 resistencias de 1 k Ω (véase Fig. 41).
- 2 capacitores de 1 μ F (véase Fig. 42).

■ **Alimentación**

- 1 fuente de 12 V de computadora (ATX).

- 1 portabaterías 3S (véase Fig. 43).
- 3 baterías recargables 18650 (véase Fig. 44).
- 1 regulador L7805 (véase Fig. 45).

■ **Conexión y cableado**

- 1 block terminal 2 vías (véase Fig. 46).
- 4 metros de cable 24 AWG, aproximadamente (señal y potencia) (véase Fig. 47).
- Termocontraíbles (varios diámetros) (véase Fig. 49).

■ **Fabricación y consumibles**

- 1 placa de cobre 10×15 cm (véase Fig. 48).
- Estaño para soldar (véase Fig. 50).
- 1 cautín (véase Fig. 51).
- 1 pistola de calor.

2.3. Diseño Estructural

Para el diseño estructural se tomó como referencia el boceto de la figura 1. A partir de eso, se realizó el listado de materiales estructurales de la sección 2.2.1, luego se procedió al modelado 3D de la planta.

El diseño se compone de una base principal (figura 18), formada por las piezas indicadas en el plano número 3 de los apéndices D.

Esta base integra un cobertor para el abanico (figura 20), cuya función es generar una abertura que permita el paso del aire hacia el tubo, garantizando una circulación óptima. Los componentes AG06 y AG07 conforman el cobertor de las baterías del sistema (figuras 23 y 24). La pieza AG05 de la figura 22 se utiliza para sostener el convertidor de alimentación por cable. El panel de control (figura 21) aloja el sistema de interacción con el usuario. Por último, se emplean conectores (figura 11) y pequeños tramos de tubo de PVC de 4 cm que se utilizan para conformar las patas de la planta.

La parte media de la planta consta de cuatro elementos:

- **Base del tubo (AG03):** Se utiliza para insertar el tubo de acrílico a presión (figura 20).
- **Tubo:** Se utiliza para introducir la esfera de estereofón y conducir el aire proveniente del abanico (figura 14).
- **Soportes:** Se utilizan para proporcionar estabilidad a la planta (figura 10).
- **Métrica (AG10):** Se utiliza como referencia visual para la altura a la que se encuentra la esfera (figura 27).

Finalmente, la parte superior de la planta está compuesta por una lámina rectangular de acrílico (figura 26), el soporte del sensor T-FLuna (figura 29) y el soporte superior del tubo (figura 28), que incorpora unas pestañas internas que impiden que la esfera de estereofón salga del sistema.

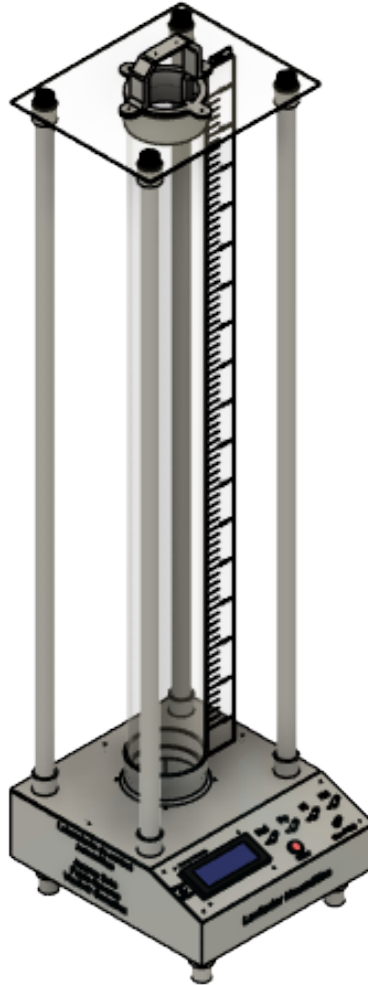


Figura 2: Diseño 3D final.

2.4. Diseño Electrónico

2.4.1. Esquemático General

La Fig. 3 muestra la arquitectura a nivel de bloques y conexiones: alimentación a 12 V (ATX o baterías 3S), regulación a 5 V mediante 7805 (con C1 y C2), sensado I²C (TF-Luna), interfaz de usuario (LCD 20×4 y potenciómetros REF/KP/KI/KD), etapa de actuación con L293D y salida al ventilador 12 V. Las entradas S1/S2 permiten modos manual, automático

y funciones auxiliares como el MAX PWM.

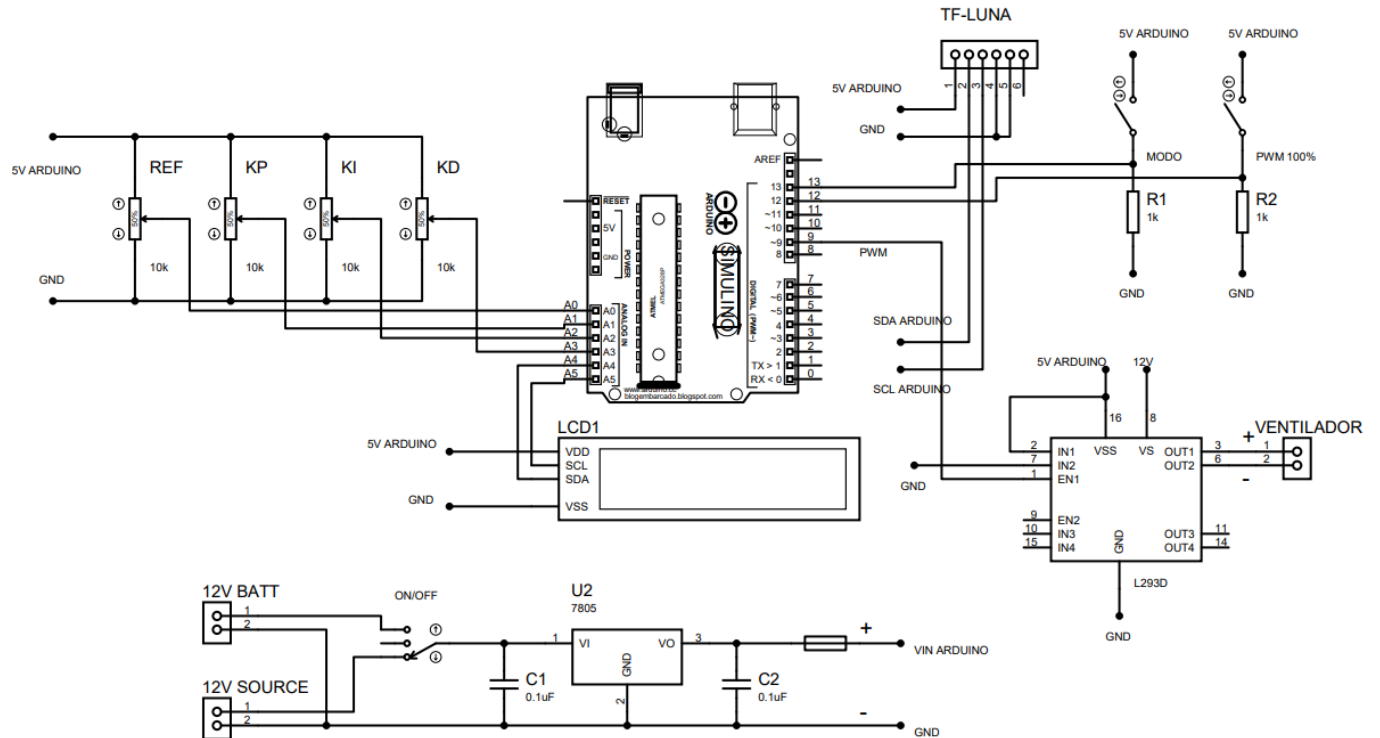


Figura 3: Esquemático general

2.4.2. Regulación y Distribución de Potencia (12 V $\rightarrow$ 5 V)

El regulador lineal 7805 (U2) suministra la lógica a 5 V para la alimentación del arduino, se recomiendan capacitores cerámicos próximos a los pines del 7805. La implementación física se muestra en la Fig. 4.

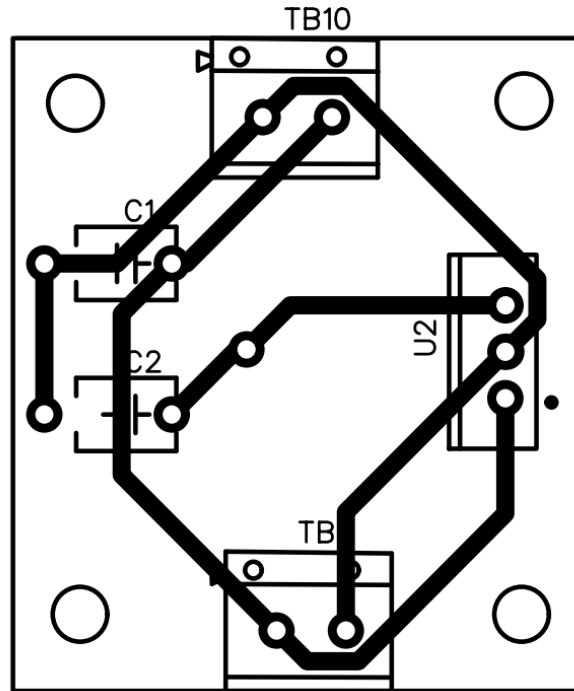


Figura 4: PCB regulador 12V $\rightarrow$ 5V

2.4.3. Actuación y Control PWM del Ventilador

El L293D conmuta el ventilador 12 V y dispone de diodos internos de rueda libre. La señal PWM proveniente pin D9 del microcontrolador modula el caudal. Se dispone de una línea “PWM 100 %” y conmutadores S0/S1/S2 para selección de modo, estas señales entran en los pines digitales D12 y D13 del microcontrolador. como se muestra en la Fig. 5.

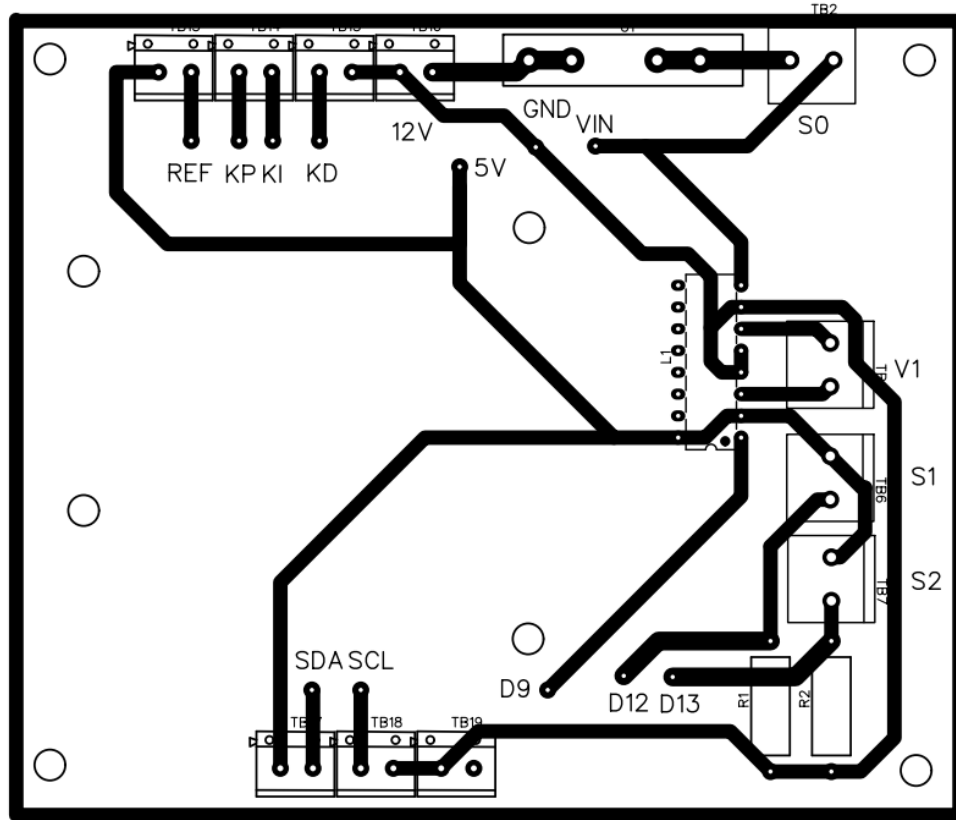


Figura 5: PCB principal

2.4.4. Sensado y Usuario

El TF-Luna se conecta por I²C (SDA/SCL) a 5 V. La LCD 20×4 (con adaptador I²C) reduce cableado y se conecta en los pines analógicos A4 y A5. Los potenciómetros REF, KP, KI y KD ingresan a entradas analógicas A0, A1, A2 y A3 para sintonía rápida en banco de pruebas.

2.4.5. Asignación de Señales

- **Alimentación:** 12 V (ATX/3S) → 7805 → 5 V lógica.
- **I²C:** SDA, SCL para TF-Luna y A4, A5 para LCD.
- **PWM ventilador:** pin digital D9 con capacidad PWM hacia L293D.
- **Entradas analógicas:** A0–A3 para REF, KP, KI, KD.
- **Entradas digitales:** S1/S2 para modo y MAX PWM en los pines D12 y D13.

3. Obtención del Modelo Empírico

En esta sección se describe el procedimiento seguido para obtener un modelo empírico de la planta de levitación neumática a partir de datos experimentales. El objetivo es disponer de un modelo que represente la dinámica dominante de la relación entre la señal de control (PWM del ventilador) y la posición de la esfera medida por el sensor TF-Luna, en torno a un punto de operación de levitación estable.

3.1. Datos experimentales

Para la identificación se realizó la toma de datos en lazo abierto aplicando un escalón a la planta. Es decir, la señal de entrada al ventilador pasó bruscamente de un valor 0 a un valor 255 de PWM, mientras se registraba la respuesta de la altura de la esfera.

Las muestras se tomaron con un periodo de muestreo constante $T_s = 0,01$ s. La respuesta medida ante dicho escalón se muestra en la Fig. 6.

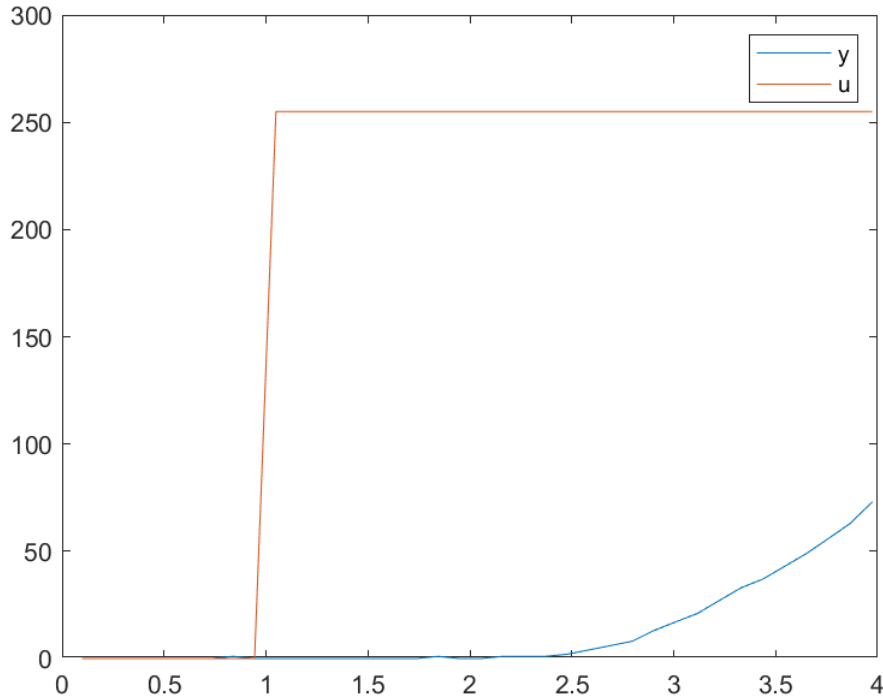


Figura 6: Respuesta de la planta ante un escalón de PWM ($0 \rightarrow 255$).

3.2. Identificación con *System Identification Toolbox*

Las señales de entrada (u) PWM y salida (y) altura se importaron a MATLAB y se procesaron con la aplicación *System Identification Toolbox*. A partir de la respuesta al escalón se probaron diferentes órdenes de modelo; finalmente se seleccionó un modelo continuo de tercer orden con un cero. El modelo continuo obtenido es la siguiente función de transferencia:

$$G(s) = \frac{0,52665 s + 10,53}{s^3 + 7,936 s^2 + 16,05 s + 2,909}. \quad (1)$$

La comparación entre la salida medida y la salida del modelo continuo identificado se muestra en la Fig. 7. La herramienta reporta un porcentaje de ajuste del 87.25 %, lo cual se considera adecuado para el diseño preliminar del controlador.

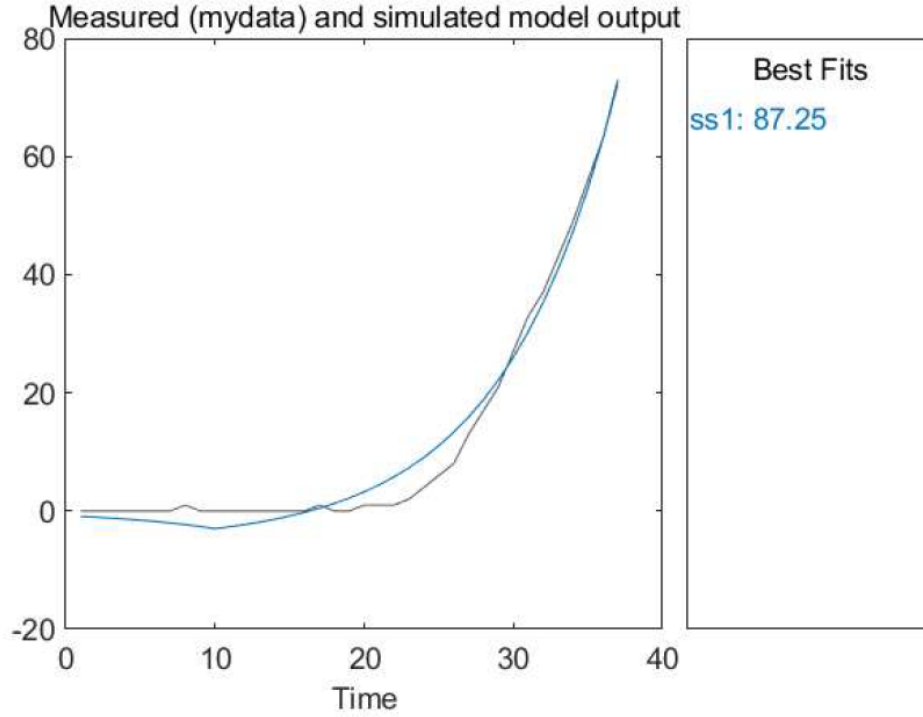


Figura 7: Comparación entre la respuesta medida y el modelo continuo 87.25 % de aproximación, SIToolbox).

3.3. Modelo discreto equivalente

Dado que la implementación del controlador se realizará en un microcontrolador, es necesario disponer de un modelo en tiempo discreto. A partir de la función de transferencia continua de la Ec. (1) y utilizando un periodo de muestreo $T_s = 0,01$ s, se obtuvo el modelo discreto mediante discretización (retención de orden cero).

La función de transferencia discreta resultante es:

$$G(z) = \frac{0,003474 z^2 + 0,004285 z - 0,0005858}{z^3 - 2,342 z^2 + 1,796 z - 0,4522}. \quad (2)$$

Este modelo discreto $G(z)$ se utilizará como base para el diseño del controlador digital y su posterior implementación en el Arduino.

4. Obtención del Controlador de la Planta

En esta sección se describe el procedimiento seguido para el diseño del controlador de la planta de levitación neumática, utilizando como punto de partida el modelo empírico obtenido

en la sección anterior y la herramienta *PID Tuner* de MATLAB.

4.1. Criterios de diseño

El objetivo del controlador es garantizar un seguimiento adecuado de referencias de posición, con buena rapidez de respuesta y oscilaciones acotadas. Como especificaciones de desempeño en lazo cerrado se plantearon las siguientes metas aproximadas:

- Tiempo de establecimiento (2 %) cercano a $t_s \approx 5,5$ s.
- Sobreimpulso máximo del orden de $OS \approx 10$ %.
- Error en estado estacionario despreciable ante entradas tipo escalón.

4.2. Sintonización con *PID Tuner*

Partiendo del modelo de la planta, se utilizó la aplicación *PID Tuner* de MATLAB para obtener un controlador de tipo PID en forma paralela. La herramienta ajustó automáticamente los parámetros para cumplir las especificaciones anteriores, generando una respuesta en lazo cerrado como la mostrada en la Fig. 8, donde se observa un sobreimpulso moderado y un tiempo de establecimiento cercano a 5,49 s.

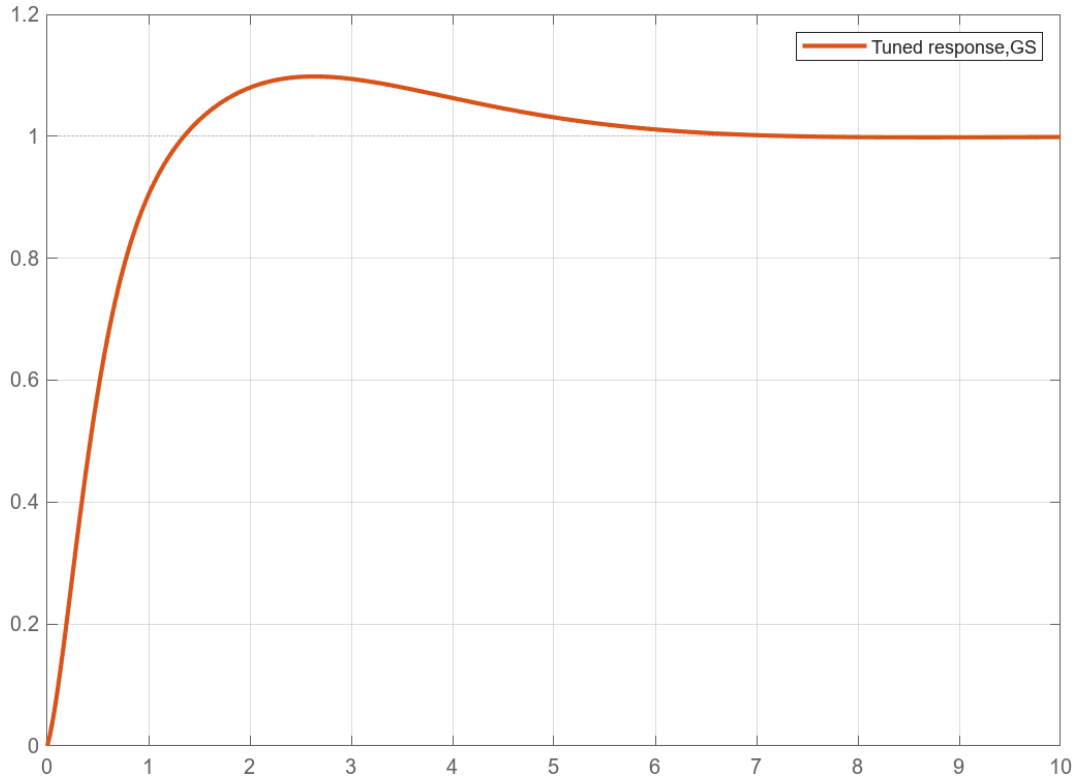


Figura 8: Respuesta al escalón del sistema en lazo cerrado con el controlador PID sintonizado en *PID Tuner* (modelo lineal).

Como punto de partida, *PID Tuner* proporcionó una ganancia proporcional $K_p = 0,3945$, una ganancia integral $K_i = 0,1268$ y una ganancia derivativa del orden de $K_d = 0,1$. Estos valores satisfacen las especificaciones en simulación sobre el modelo lineal de la planta.

4.3. Ajustes prácticos en la planta real

Durante las pruebas experimentales se observó que el ventilador presenta una no linealidad importante: existe un valor mínimo de PWM a partir del cual comienza a generar un flujo de aire suficiente para levantar la esfera. Esta *zona muerta* del actuador no está recogida en el modelo lineal identificado.

Con las ganancias originales sugeridas por *PID Tuner*, la señal de control en lazo cerrado permanecía durante un intervalo significativo por debajo de dicho umbral, por lo que la esfera no llegaba a despegar del fondo del tubo. Fue necesario incrementar la intensidad del controlador para que el mando PWM superara de forma consistente el valor mínimo requerido por el ventilador y la planta entrara en la zona de operación de interés.

Los valores finales del controlador, así como la respuesta resultante, se resumen en la subsección siguiente.

4.4. Controlador resultante

Con base en las observaciones anteriores, se decidió escalar las ganancias obtenidas en simulación:

- La ganancia proporcional se incrementó mediante un factor de 7, con el fin de obtener una respuesta más rápida y garantizar que el mando supere la zona muerta del ventilador:

$$K_p = 0,3945 \times 7 = 2,7615.$$

- La ganancia integral se incrementó mediante un factor de 40 respecto al valor base recomendado por *PID Tuner*, con el objetivo de reducir más rápidamente el error en estado estacionario:

$$K_i = 40 \times K_{i0} = 40 \times 0,1268 \approx 5,072.$$

En MATLAB esta elección se implementó de forma equivalente mediante los tiempos del controlador:

$$T_i = \frac{K_p}{0,1268 \times 40} \approx 0,5445 \text{ s}, \quad T_d = \frac{0,1}{K_p} \approx 0,0362 \text{ s},$$

de modo que:

$$K_i = \frac{K_p}{T_i} \approx 5,072, \quad K_d = K_p T_d \approx 0,10.$$

El controlador continuo de tipo PID en forma paralela, con los valores finalmente adoptados, queda definido como:

$$C(s) = K_p + \frac{K_i}{s} + K_d s = 2,7615 + \frac{5,072}{s} + 0,10 s. \quad (3)$$

Alternativamente, en términos de los tiempos T_i y T_d :

$$C(s) = K_p \left(1 + \frac{1}{T_i s} + T_d s \right), \quad (4)$$

donde $K_p = 2,7615$, $T_i \approx 0,5445$ s y $T_d \approx 0,0362$ s.

La implementación de este controlador escalado en la planta real produce una respuesta al escalón como la mostrada en la Fig. 9, en la cual se aprecia un sobreimpulso elevado y oscilaciones amortiguadas antes de alcanzar el valor de referencia. Este comportamiento refleja el compromiso entre lograr que el sistema arranque efectivamente (venciendo la zona muerta del actuador) y mantener un lazo de control fuertemente amortiguado.

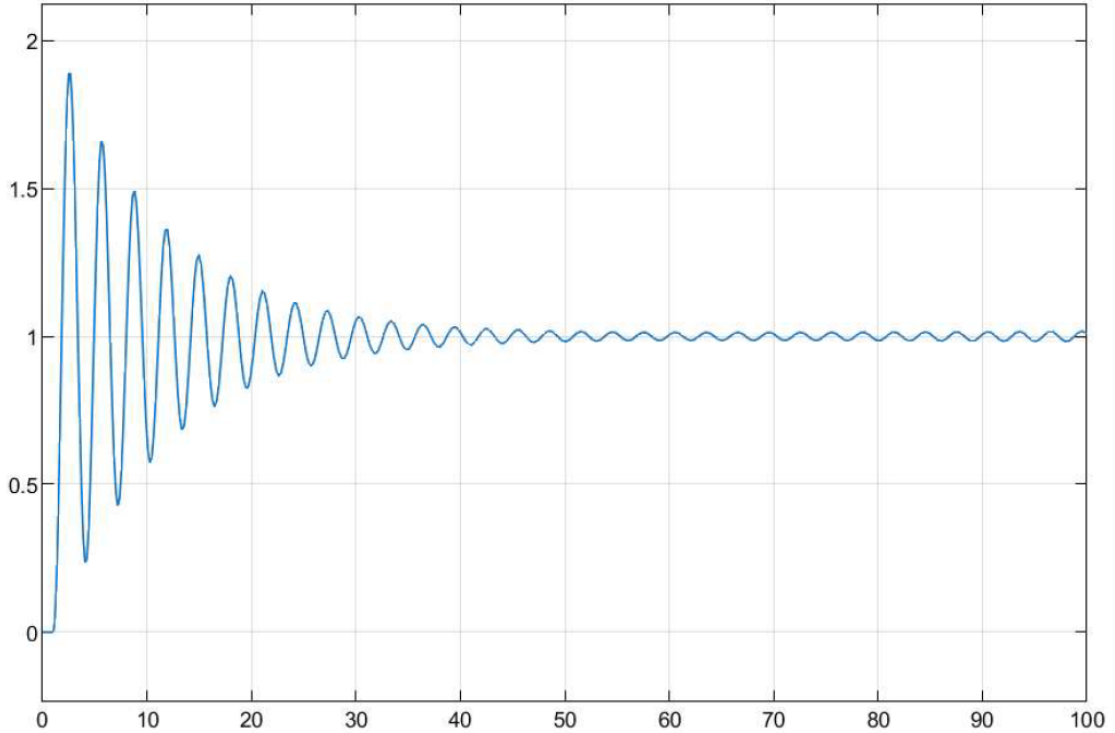


Figura 9: Respuesta al escalón del sistema en lazo cerrado con el controlador PID escalado implementado en la planta real.

5. Controlador Digital

El controlador diseñado en tiempo continuo se implementó en el Arduino Uno mediante una versión discreta en forma incremental, usando un tiempo de muestreo de $T_s = 0,01$ s. En esta sección se muestran únicamente los fragmentos del código relevantes para el cálculo de la ley de control y la generación de la señal PWM hacia el ventilador.

5.1. Variables y estructura del PID incremental

El controlador se implementa en forma incremental,

$$u[k] = u[k-1] + A e[k] + B e[k-1] + C e[k-2],$$

almacenando las salidas previas y los errores anteriores:

```
1 double uk = 0.0;    // u(k)
2 double ek = 0.0;    // e(k)
3 double ukm1 = 0.0;  // u(k-1)
4 double ukm2 = 0.0;  // u(k-2)
5 double ekm1 = 0.0;  // e(k-1)
6 double ekm2 = 0.0;  // e(k-2)
7
8 float Ts = 0.01f;   // Tiempo de muestreo
9
10 // Coeficientes del PID incremental
11 double A, B, C;
```

Listing 1: Variables principales del controlador incremental.

A partir de K_p , T_i y T_d (en modo automático o manual), los coeficientes se calculan con:

```
1 A = kp * (1 + (Ts/(2*ti)) + (td/Ts));
2 B = -kp * (1 - (Ts/(2*ti)) + (2*td/Ts));
3 C = kp * td / Ts;
```

Listing 2: Cálculo de los coeficientes A, B y C del PID incremental.

5.2. Modo automático: ganancias fijas

En el modo automático (`mode = 1`), el controlador usa los valores de K_p , T_i y T_d definidos a partir del diseño con *PID Tuner* y los factores de escala. La referencia se obtiene de un potenciómetro y la salida de la planta del sensor TF-Luna:

```
1 if (mode == 1) {
2     float Ts = 0.01f;
3     float kp = 0.3945f * 7.0f;
4     float ti = kp / (0.1268f * 40.0f);
5     float td = 0.1f / kp;
6
7     A = kp * (1 + (Ts/(2*ti)) + (td/Ts));
8     B = -kp * (1 - (Ts/(2*ti)) + (2*td/Ts));
9     C = kp * td / Ts;
10
11     // Referencia en cm
12     float reference = analogRead(SETPOINT_PIN);
13     ref_cm = map(reference, 0, 1024, 0, 81);
14
15     // Medicin de altura con TF-Luna
```

```

16  if (tf1I2C.getData(tfDist, tfAddr)) {
17      if (tfDist < 50) Distancia = 85 - tfDist;
18      else                Distancia = 87 - tfDist;
19
20      xk = Distancia;
21      ek = ref_cm - xk;
22
23      // PID incremental
24      uk = ukm1 + (A*ek) + (B*ekm1) + (C*ekm2);
25
26      // Saturaci n PWM
27      if (uk > 255) uk = 255;
28      if (uk < 0) uk = 0;
29
30      int pwm = (int)floor(uk);
31      analogWrite(PWM_PIN, pwm);
32
33      // Actualizaci n de estados
34      ekm2 = ekm1; ekm1 = ek;
35      ukm2 = ukm1; ukm1 = uk;
36  }
37 }

```

Listing 3: Implementaci3n del PID incremental en modo autom3tico.

Este bloque representa el lazo b3sico de control en lazo cerrado: lectura de referencia, medici3n de posici3n, c3lculo de la ley incremental y env3o de la se3al PWM al ventilador.

5.3. Modo manual: ajuste experimental de K_p , K_i y K_d

En modo manual (`mode = 0`), las ganancias se ajustan mediante tres potenc3metros, lo que permite explorar distintas sinton3as sobre la planta real:

```

1  if (mode == 0) {
2      int a_kp = analogRead(KP);
3      int a_ki = analogRead(KI);
4      int a_kd = analogRead(KD);
5
6      ref_cm = analogRead(SETPOINT_PIN) * (81.0f / 1023.0f);
7
8      kp_r = a_kp * (10.0f / 1023.0f);
9      ki_r = a_ki * (15.0f / 1023.0f);
10     kd_r = a_kd * (0.50f / 1023.0f);
11
12     if (kp_r < 0.05f) kp_r = 0.05f;
13     if (ki_r < 0.05f) ki_r = 0.05f;
14
15     float Ts = 0.01f;
16     float kp = kp_r;

```

```

17 float ti = kp_r / ki_r;
18 float td = (kp_r > 0) ? (kd_r / kp_r) : 0.0f;
19
20 if (ti < 0.15f) ti = 0.15f;
21 if (ti > 2.00f) ti = 2.00f;
22 if (td < 0.0f) td = 0.0f;
23 if (td > 0.08f) td = 0.08f;
24
25 A = kp * (1 + (Ts/(2*ti)) + (td/Ts));
26 B = -kp * (1 - (Ts/(2*ti)) + (2*td/Ts));
27 C = kp * td / Ts;
28 }

```

Listing 4: Lectura de potenciómetros y cálculo de A, B y C en modo manual.

Una vez calculados A , B y C , el resto del lazo de control (lectura del TF–Luna, cálculo de $e[k]$, evaluación de la ecuación incremental y saturación de $u[k]$) es análogo al modo automático.

6. Resultados

En esta sección se presentan los resultados experimentales obtenidos con el sistema de levitación neumática ya construido e instrumentado, empleando el controlador PID descrito en las secciones anteriores. Se consideran principalmente las pruebas en modo automático (ganancias fijas escaladas) y algunas observaciones relevantes durante la operación en modo manual. Además del análisis cuantitativo presentado, el comportamiento del sistema puede apreciarse de forma cualitativa en el video demostrativo incluido en el Apéndice A.

6.1. Pruebas de seguimiento de referencia en modo automático

La primera serie de pruebas consistió en aplicar escalones de referencia de altura en modo automático, partiendo desde la esfera apoyada en la base del tubo y fijando un valor de referencia dentro del rango operativo (aproximadamente entre 30 y 60 cm). El controlador utiliza las ganancias continuas de la Ec. (3), discretizadas mediante la formulación incremental implementada en el Arduino.

La Fig. 8 muestra la respuesta prevista por el modelo lineal en *PID Tuner*, con un sobreimpulso cercano al 10 % y un tiempo de establecimiento de alrededor de 5,5 s. Sin embargo, al implementar el controlador en la planta real y escalar las ganancias según se describe en la Sección 4, se obtuvo una respuesta como la mostrada en la Fig. 9.

En esta respuesta experimental se aprecia que:

- El sistema logra finalmente alcanzar y mantener la altura de referencia, cumpliendo el objetivo de levitar la esfera en una posición estable dentro del tubo.
- El sobreimpulso es significativamente mayor que el observado en simulación, del orden de varias decenas de por ciento, con oscilaciones amortiguadas alrededor del valor de referencia.

- El tiempo de establecimiento se incrementa ligeramente respecto al valor nominal, debido a las oscilaciones que aparecen tras el primer pico de la respuesta.

Aun con estas diferencias, el comportamiento en lazo cerrado es estable y permite demostrar visualmente el concepto de control de posición mediante levitación neumática.

6.2. Funcionamiento en modo manual

En modo manual, las ganancias K_p , K_i y K_d se ajustan mediante potenciómetros conectados a las entradas analógicas del Arduino. Este modo se utilizó para explorar sintonías alternativas y observar en tiempo real el efecto de cada parámetro sobre la dinámica de la levitación.

Durante las pruebas se observaron los siguientes comportamientos típicos:

- Un incremento de K_p mejora la rapidez de respuesta, pero también aumenta el riesgo de oscilaciones marcadas y sobreimpulso.
- Un valor muy bajo de K_i produce errores en estado estacionario apreciables, mientras que un valor demasiado alto introduce oscilaciones lentas y tendencia al *windup* de la acción integral.
- La acción derivativa (K_d) ayuda a amortiguar la respuesta cuando se seleccionan ganancias proporcionales relativamente altas, pero también amplifica el ruido si se aumenta en exceso.

El modo manual resultó especialmente útil para validar la influencia de las ganancias sobre la planta real y para encontrar, de forma experimental, combinaciones de parámetros que facilitan el arranque de la levitación sin comprometer la estabilidad.

7. Análisis de Resultados

Los resultados obtenidos permiten comparar el comportamiento ideal previsto en simulación con la respuesta real de la planta neumática. En esta sección se discuten las principales diferencias y sus causas probables.

7.1. Modelo lineal vs. planta real

El modelo continuo de tercer orden identificado en la Sección 3 presenta un ajuste del 87.25 % respecto a los datos experimentales de identificación. Este nivel de ajuste es razonable para capturar la dinámica dominante, pero deja fuera varias no linealidades presentes en la planta real:

- **Zona muerta del ventilador:** el ventilador de 12 V no genera un flujo de aire útil por debajo de cierto valor de PWM. Esto implica que, aunque el controlador incrementa gradualmente la señal de mando, la esfera no empezará a levitar hasta que el PWM supere ese umbral.

- **Saturación de la etapa de potencia:** la señal de control está limitada al rango 0–255. Cuando el controlador integral acumula error, la salida puede saturarse y producir sobreimpulsos más severos al abandonar la saturación.
- **Efectos aerodinámicos no lineales:** la relación entre caudal de aire, posición de la esfera y pérdidas por turbulencia en el tubo acrílico es fuertemente no lineal y dependiente de la altura.

Estas características explican por qué el comportamiento real presenta un sobreimpulso y oscilaciones mayores que las predichas por el modelo lineal utilizado en *PID Tuner*.

7.2. Impacto del escalado de las ganancias

El escalado aplicado a K_p y K_i tuvo como motivación principal asegurar que el PWM alcanzara valores suficientes para vencer la zona muerta del ventilador y arrancar la levitación desde la esfera en reposo. Esto efectivamente se consiguió: con las ganancias sin escalar, el sistema podía estabilizarse si ya estaba en operación, pero no siempre lograba despegar desde el fondo del tubo.

El costo de este escalado es:

- Un aumento considerable en el sobreimpulso de la respuesta al escalón.
- Oscilaciones amortiguadas alrededor de la referencia, más visibles que en la simulación.
- Mayores exigencias a la etapa de potencia, ya que se trabaja más cerca de los valores máximos de PWM.

No obstante, desde el punto de vista pedagógico, estos parámetros permiten visualizar claramente el fenómeno de levitación y el efecto de la realimentación, lo cual era uno de los objetivos del proyecto.

7.3. Efectos de la implementación digital

La discretización del controlador y su implementación en Arduino también introducen efectos adicionales:

- El periodo de muestreo $T_s = 0,01\text{s}$ es suficientemente pequeño para la dinámica de la planta, pero el uso de `delay()` y la ejecución de tareas de impresión en la LCD introducen retardos efectivos que no están modelados en el diseño continuo.
- La cuantización del PWM (8 bits) y de las mediciones (resolución del ADC y del sensor TF–Luna) puede generar pequeñas oscilaciones alrededor del valor de referencia, especialmente cuando el sistema se acerca al régimen estacionario.
- El uso de la forma incremental del PID ayuda a evitar errores numéricos y facilita el manejo de saturaciones, pero también hace más sensible el controlador a errores en los coeficientes A , B y C si se modifican las ganancias en tiempo real sin una lógica de re-inicialización adecuada.

En conjunto, estos factores explican la diferencia entre la respuesta «limpia» obtenida en *PID Tuner* y la respuesta más oscilatoria observada en la planta real, aun cuando el sistema se mantiene estable y capaz de seguir la referencia con error en régimen aceptable.

7.4. Utilidad del modo manual

El modo manual, con ajuste de K_p , K_i y K_d mediante potenciómetros, se reveló como una herramienta valiosa para:

- Explorar el efecto de cada ganancia sobre la dinámica real de la planta, más allá de los resultados del modelo lineal.
- Encontrar regiones de operación donde la respuesta es más suave (por ejemplo, reduciendo ligeramente K_p y K_i cuando la esfera ya está levitando).
- Ilustrar de forma didáctica el compromiso clásico entre rapidez, sobreimpulso y robustez en sistemas de control realimentado.

Este modo complementa el diseño teórico, permitiendo al usuario «afinar» la sintonía directamente en el hardware y visualizar inmediatamente el efecto de los cambios.

8. Conclusiones

El proyecto desarrollado permitió diseñar, construir e implementar un sistema de levitación neumática controlado mediante un microcontrolador Arduino Uno y un controlador PID digital. En términos generales, el sistema fue capaz de mantener una esfera de estereofón levitando dentro de un tubo acrílico y de seguir cambios en la referencia de altura dentro de un rango operativo definido. Aunque la respuesta presenta sobreimpulsos y oscilaciones más pronunciadas de lo esperado en el diseño lineal, el comportamiento en lazo cerrado es estable y cumple el objetivo principal de demostrar el control de posición mediante levitación neumática.

El modelo empírico obtenido a partir de datos experimentales, utilizando la herramienta *System Identification Toolbox* de MATLAB, proporcionó una base razonable para el diseño inicial del controlador PID. Con un ajuste cercano al 87.25 % respecto a la respuesta medida, el modelo continuo de tercer orden logró capturar la dinámica dominante del sistema. Sin embargo, la comparación con la planta real mostró claramente las limitaciones de este enfoque lineal: la presencia de zona muerta en el ventilador, la saturación de la señal PWM y la naturaleza no lineal del flujo de aire dentro del tubo introducen efectos que no están recogidos en la aproximación obtenida.

Una de las conclusiones más relevantes es la importancia de considerar explícitamente las no linealidades del actuador. En la práctica, fue necesario escalar de forma significativa las ganancias proporcional e integral del controlador para garantizar que la señal de control superara el umbral mínimo de activación del ventilador y permitiera el arranque de la levitación desde la esfera en reposo. Este ajuste mejoró la capacidad del sistema para entrar en la zona de operación deseada, pero a costa de incrementar el sobreimpulso y las oscilaciones

transitorias. El resultado ilustra de manera clara el compromiso clásico entre robustez frente a efectos no lineales y calidad de la respuesta transitoria.

La implementación digital del controlador en el Arduino, empleando una formulación incremental con un tiempo de muestreo de $T_s = 0,01$ s, demostró ser adecuada para el control en tiempo real. No obstante, se evidenció que detalles prácticos como los retardos introducidos por el uso de `delay()` y la actualización de la pantalla LCD, la cuantización del PWM y del ADC, así como el ruido en la medición del sensor, tienen un impacto perceptible en la respuesta real del sistema. Estos aspectos ponen de manifiesto la diferencia entre un diseño ideal continuo y su implementación embebida en hardware con recursos limitados.

Finalmente, el modo manual de operación, que permite ajustar K_p , K_i y K_d mediante potenciómetros, aportó un valor didáctico significativo. Este modo facilitó la exploración de distintas sintonías directamente sobre la planta real y permitió observar en tiempo real el efecto de los cambios de ganancia sobre la dinámica de la levitación. De esta forma, el sistema no solo cumple como demostrador de control en lazo cerrado, sino también como herramienta de enseñanza para comprender de manera tangible la influencia de los parámetros de un controlador PID y las diferencias entre la teoría y la práctica en sistemas de control reales.

Como trabajo futuro, resultaría de interés incorporar estrategias de compensación de zona muerta y técnicas de anti-*windup* para la acción integral, con el objetivo de reducir el sobreimpulso y mejorar el rechazo de perturbaciones. Asimismo, podría considerarse el uso de controladores más avanzados o basados en modelos no lineales, así como la optimización de la implementación digital evitando retardos innecesarios y registrando datos de forma sistemática para un análisis más detallado del desempeño del sistema.

9. Apéndices

A. Video de funcionamiento del sistema

El funcionamiento del sistema de levitación neumática puede observarse en el siguiente video demostrativo:

- **Video en YouTube:** https://www.youtube.com/watch?v=e_gDNlKtYk

B. Imágenes de componentes estructurales



Figura 10: Tubo PVC eléctrico 1/2 [3].



Figura 11: Conector PVC 1/2 [3].



Figura 12: Tapón PVC 1/2 [3].



Figura 13: Lámina acrílica transparente 3 mm [4].



Figura 14: Tubo redondo rígido de acrílico [4].



Figura 15: Rollo de PLA negro 1.75 mm, 1 kg [5].

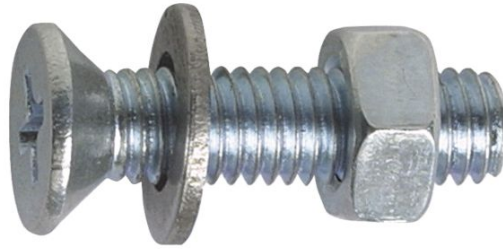


Figura 16: Tornillo, tuerca y arandela [3].



Figura 17: Bola de estereofón (EPS) [2].

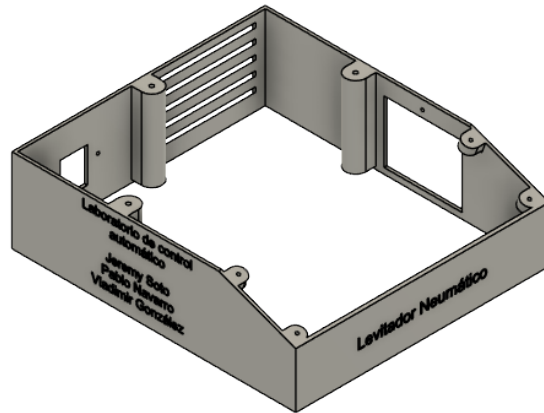


Figura 18: Caja base.

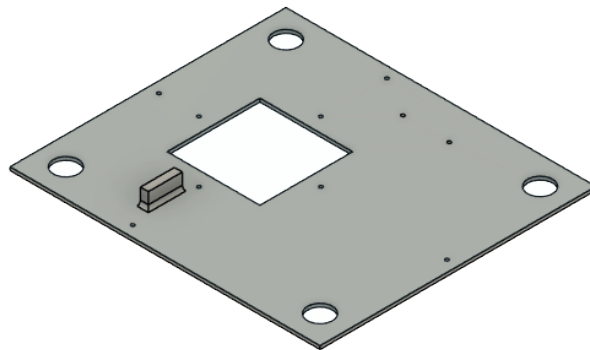


Figura 19: Tapa base.

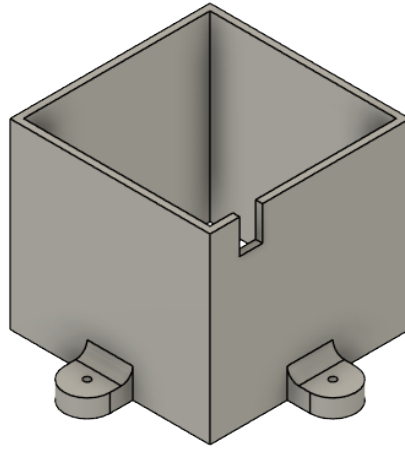


Figura 20: Cobertor de abanico.

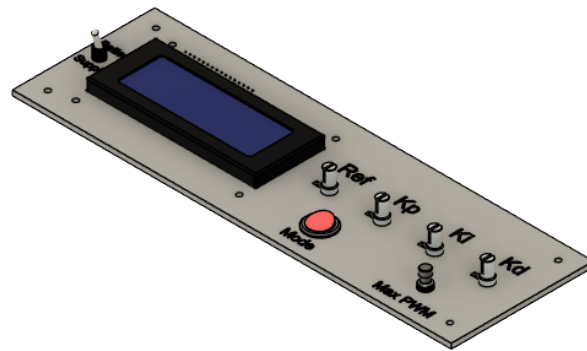


Figura 21: Panel de control.

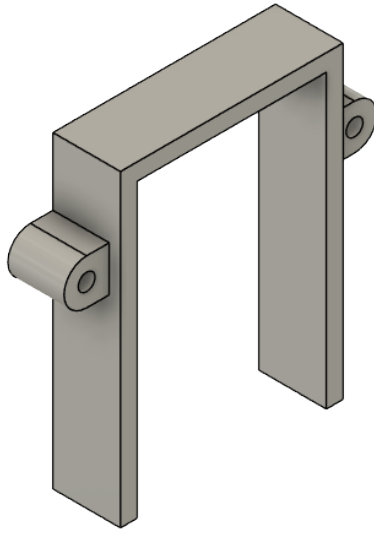


Figura 22: Soporte de convertidor.

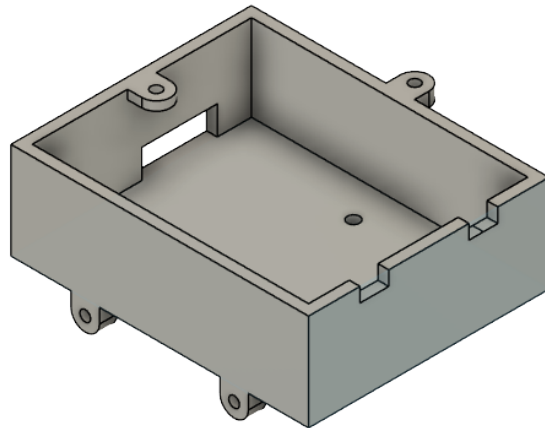


Figura 23: Caja de baterías.

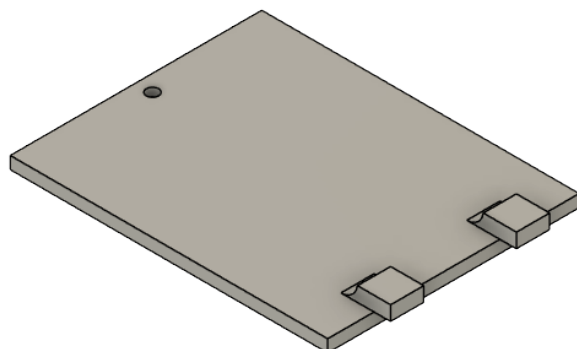


Figura 24: Tapa de baterías.

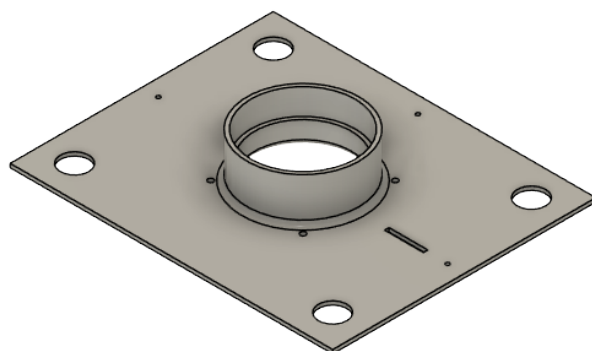


Figura 25: Tapa inferior.

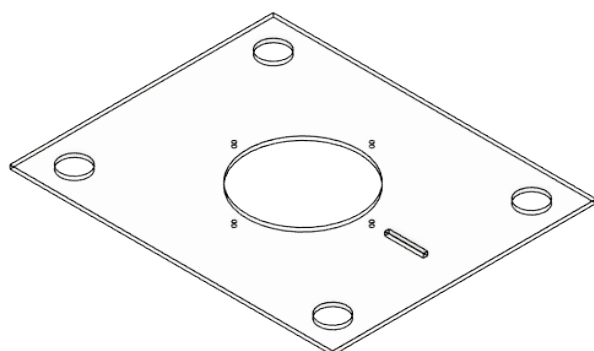


Figura 26: Placa superior.



Figura 27: Métrica.

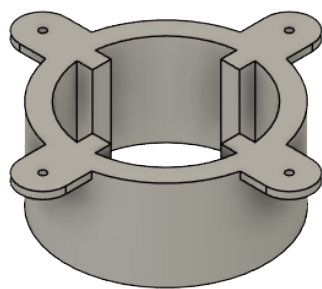


Figura 28: Soporte superior del tubo.

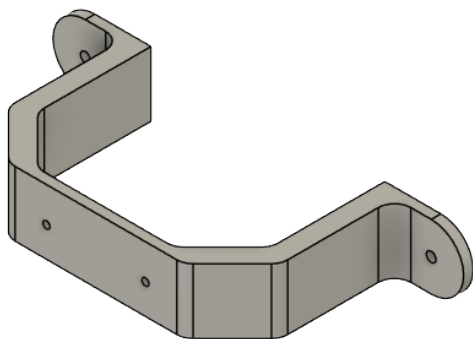


Figura 29: Soporte del sensor.

C. Imágenes de componentes electrónicos



Figura 30: Arduino Uno [5]

Figura 31: LCD 20×4 I2C [5]

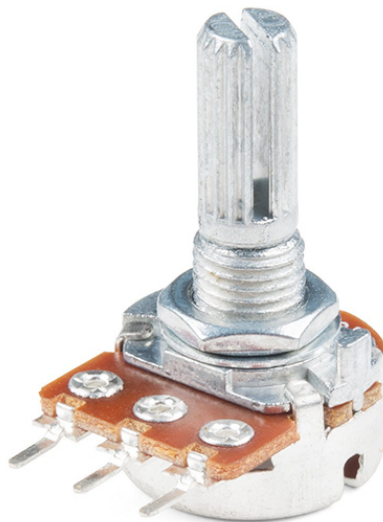


Figura 32: Potenciómetro 10 k Ω [5]



Figura 33: Pulsador [5]



Figura 34: Switch redondo [5]



Figura 35: Switch de 3 estados [5]



Figura 36: Conector macho (pines) para Arduino [5]



Figura 37: Jumpers hembra-macho [5]



Figura 38: Sensor TF-Luna



Figura 39: Ventilador 12 V 80×80×25 mm

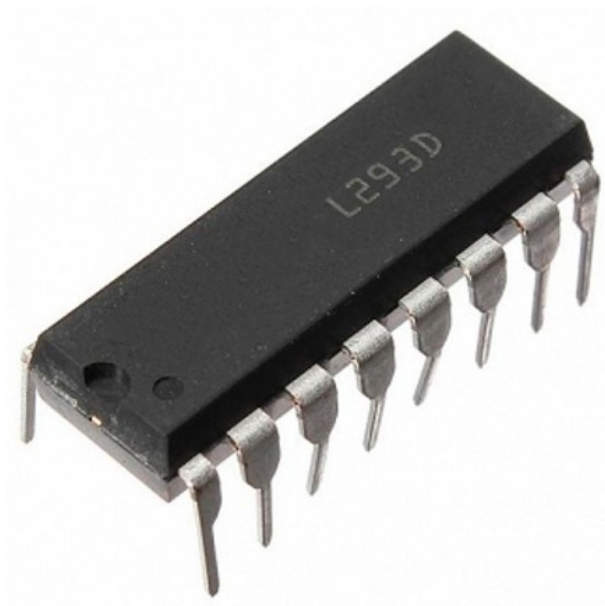


Figura 40: Puente H L293D [5]



Figura 41: Resistencia 1 k Ω [5]

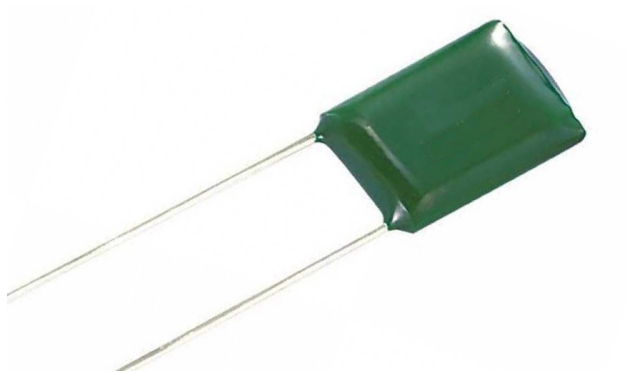


Figura 42: Capacitor cerámico 1 μ F [5]



Figura 43: Portabaterías 18650 (3S)



Figura 44: Baterías recargables 18650



Figura 45: Regulador L7805 [5]



Figura 46: Block terminal de 2 vías [5]



Figura 47: Cable 24 AWG



Figura 48: Placa de cobre 10×15 cm [5]



Figura 49: Termocontraíble

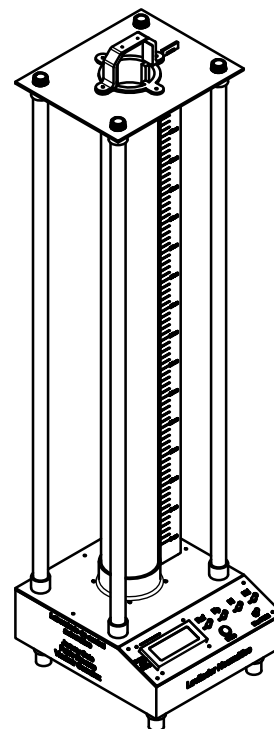
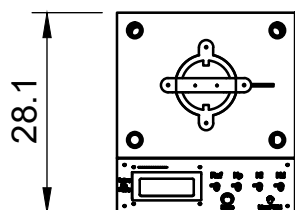
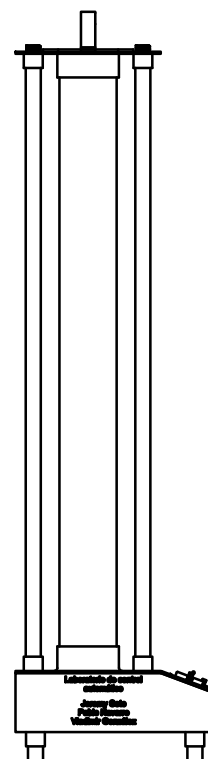
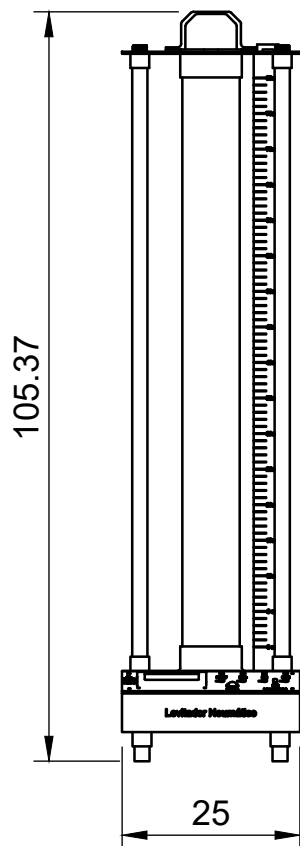


Figura 50: Estaño para soldar



Figura 51: Cautín

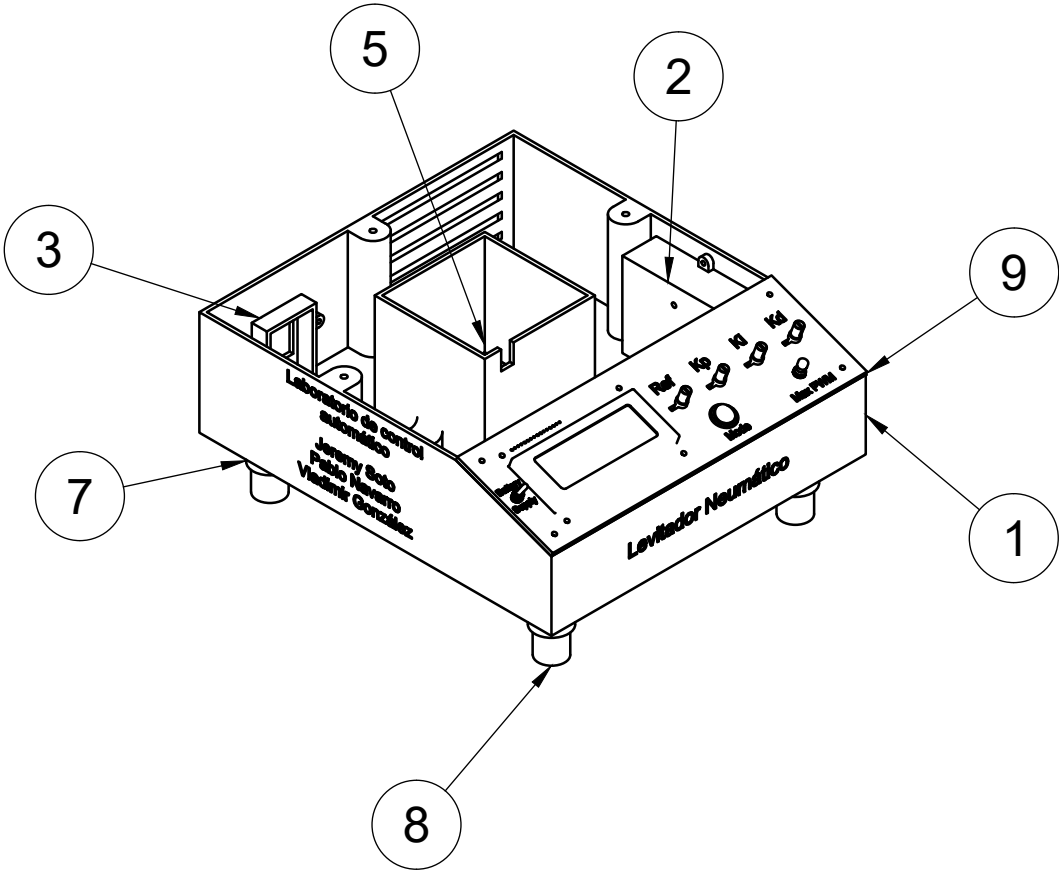
D. Planos de la planta



Dept. I.E	Technical reference	Created by ANTIGRAVITY 23/10/2025	Approved by Pablo Cesar Rodriguez 13/11/2025	
		Document type Plano	Document status	
		Title ENSAMBLE Unidades: cm	DWG No.	
			Rev.	Date of issue
			Sheet 1/3	

Parts List		
Elem ento	Canti dad	Nombre de pieza
1	1	AG01
2	1	AG06
3	1	AG05
4	1	AG02
5	1	AG03
6	1	AG07
7	12	Conector pvc
8	4	Patas
9	1	AG04
10	1	AG08
11	1	Tubo
12	4	Soporte
13	1	AG11
14	1	AG09
15	1	AG12
16	1	AG10

Dept. I.E	Technical reference	Created by ANTIGRAVITY 23/10/2025	Approved by Pablo Cesar Rodriguez 13/11/2025		
			Document type Plano		
			Title ENSAMBLE Lista de piezas		
			Document status		
			DWG No.		
			Rev.	Date of issue	Sheet 2/3

				Parts List		
				Elem ento	Canti dad	Nombre de pieza
				1	1	AG01
				2	1	AG06
				3	1	AG05
				4	1	AG02
				5	1	AG03
				6	1	AG07
				7	4	Conector pvc
				8	4	Patas
				9	1	AG04
						
Dept. I.E	Technical reference	Created by ANTIGRAVITY 23/10/2025		Approved by Pablo Cesar Rodriguez 13/11/2025		
		Document type Plano		Document status		
		Title ENSAMBLE Lista de piezas internas		DWG No.		
				Rev.	Date of issue	Sheet 3/3

E. Codigó de la planta

```
1 #include <Arduino.h>
2 #include <Wire.h>
3 #include <TFLI2C.h>
4 #include <LiquidCrystal_I2C.h>
5 #include <math.h>
6
7 TFLI2C tflI2C;
8 LiquidCrystal_I2C lcd(0x27, 20, 4);
9
10 #define D12_PIN          12
11 #define D13_PIN          13
12 #define PWM_PIN          9
13 #define SETPOINT_PIN     A3
14 #define KP                A2
15 #define KI                A1
16 #define KD                A0
17
18 const float ADC_MAX = 1023.0f;
19 const unsigned long INTERVALO_MUESTREO_MS = 10;
20
21 double ref_cm = 0.0;
22 float kp_r = 0.0;
23 float ki_r = 0.0;
24 float kd_r = 0.0;
25
26 int16_t tfDist;
27 int16_t tfAddr = TFL_DEF_ADR;
28 int     Distancia;
29
30 double uk=0, xk=0, ek=0;
31 double ukm1=0, xkm1=0, ukm2=0, xkm2=0;
32 double ekm1=0, ekm2=0;
33
34 int i=0;
35 float Ts=0.01;
36 float kp=3.6;
37 float ti=kp/6;
38 float td=0.09/kp;
39 double A, B, C;
40 int pwm;
41 float reference;
42 float ref_kp, ref_ki, ref_kd;
43 int mode;
44 int max_pwm;
45
```

```

46 void printLine(uint8_t row, const String &text) {
47     lcd.setCursor(0, row);
48     lcd.print("░░░░░░░░░░░░░░░░░░░░");
49     lcd.setCursor(0, row);
50     lcd.print(text);
51 }
52
53 void setup() {
54     A = kp*(1+(Ts/(2*ti))+(td/Ts));
55     B = -kp*(1-(Ts/(2*ti))+(2*td/Ts));
56     C = kp*td/Ts;
57
58     Serial.begin(115200);
59     Wire.begin();
60
61     pinMode(D12_PIN, INPUT);
62     pinMode(D13_PIN, INPUT);
63     pinMode(PWM_PIN, OUTPUT);
64
65     lcd.init();
66     lcd.backlight();
67     lcd.clear();
68     printLine(0, "░░***BIENVENIDO***░");
69     printLine(1, "░░░░░Levitador");
70     printLine(2, "░░░░░Neumatico");
71     printLine(3, "░░░░░░░░░░░░░░░-TEC");
72     delay(4000);
73
74     lcd.clear();
75     printLine(0, "░░░░*Hecho░por:░");
76     printLine(1, "-Jeremy░Soto");
77     printLine(2, "-Pablo░Navarro");
78     printLine(3, "-Vladimir░Gonzalez");
79     delay(4000);
80 }
81
82 void loop() {
83     mode = digitalRead(D13_PIN);
84     max_pwm = digitalRead(D12_PIN);
85
86     if (max_pwm == 1) {
87         analogWrite(PWM_PIN, 255);
88         printLine(0, "░░░░░░░░░░░░░░░░░░░░");
89         printLine(1, "-MAX_PWM");
90         printLine(2, "░░░░░░░░░░░░░░░░░░░░");
91         printLine(3, "░░░░░░░░░░░░░░░░░░░░");
92     } else {
93         if (mode == 1) {

```

```

94     float Ts=0.01;
95     float kp=0.3945*7;
96     float ti=kp/(0.1268*40);
97     float td=0.1/kp;
98
99     A = kp*(1+(Ts/(2*ti))+(td/Ts));
100    B = -kp*(1-(Ts/(2*ti))+(2*td/Ts));
101    C = kp*td/Ts;
102
103    reference = analogRead(SETPOINT_PIN);
104    ref_cm = map(reference, 0, 1024, 0, 81);
105
106    if (tfII2C.getData(tfDist, tfAddr)) {
107        if (tfDist < 50) Distancia = 85 - tfDist;
108        else Distancia = 87 - tfDist;
109
110        xk = Distancia;
111        ek = ref_cm - xk;
112        uk = ukm1 + (A*ek) + (B*ekm1) + (C*ekm2);
113
114        if (uk >= 255) uk = 255;
115        if (uk <= 0) uk = 0;
116        pwm = floor(uk);
117        analogWrite(PWM_PIN, pwm);
118
119        printLine(0, "Modo automatico");
120        printLine(1, "Sensor:");
121        printLine(2, "Sensor:" + String(Distancia) + "cm");
122        printLine(3, "Ref:" + String(ref_cm, 0) + "cm");
123        delay(INTERVALO_MUESTREO_MS);
124
125        if (i>=2) {
126            xkm2=xkm1;
127            ukm2=ukm1;
128            ekm2=ekm1;
129        }
130        xkm1 = xk;
131        ukm1 = uk;
132        ekm1 = ek;
133        i += 1;
134    }
135 }
136
137 if (mode == 0) {
138     int a_kp = analogRead(KP);
139     int a_ki = analogRead(KI);
140     int a_kd = analogRead(KD);
141

```

```

142     ref_cm = analogRead(SETPOINT_PIN) * (81.0f / 1023.0f);
143
144     kp_r = a_kp * (10.0f / 1023.0f);
145     ki_r = a_ki * (15.0f / 1023.0f);
146     kd_r = a_kd * (0.50f / 1023.0f);
147
148     if (kp_r < 0.05f) kp_r = 0.05f;
149     if (ki_r < 0.05f) ki_r = 0.05f;
150
151     float Ts = 0.01f;
152     float kp = kp_r;
153     float ti = kp_r / ki_r;
154     float td = kd_r/kp_r;
155
156     if (ti < 0.15f) ti = 0.15f;
157     if (ti > 2.00f) ti = 2.00f;
158     if (td < 0.0f) td = 0.0f;
159     if (td > 0.08f) td = 0.08f;
160
161     A = kp * (1 + (Ts/(2*ti)) + (td/Ts));
162     B = -kp * (1 - (Ts/(2*ti)) + (2*td/Ts));
163     C = kp * td / Ts;
164
165     if (isnan(A) || isnan(B) || isnan(C) ||
166         isinf(A) || isinf(B) || isinf(C)) {
167         A = 0; B = 0; C = 0;
168     }
169
170     if (tfllI2C.getData(tfDist, tfAddr)) {
171         Distancia = (tfDist < 50) ? (85 - tfDist) : (87 - tfDist);
172
173         xk = Distancia;
174         ek = ref_cm - xk;
175
176         uk = ukm1 + (A*ek) + (B*ekm1) + (C*ekm2);
177
178         if (isnan(uk) || isinf(uk)) uk = ukm1;
179         if (uk > 255) uk = 255;
180         if (uk < 0) uk = 0;
181
182         pwm = (int)floor(uk + 0.5);
183         analogWrite(PWM_PIN, pwm);
184
185         ekm2 = ekm1; ekm1 = ek;
186         ukm2 = ukm1; ukm1 = uk;
187         xkm2 = xkm1; xkm1 = xk;
188
189         printLine(0, "UUUU-Modo_manualUU");

```

```

190     printLine(1, "Ref:" + String(ref_cm, 0) + "cm" +
191                 "Sensor:" + String(Distancia) + "cm");
192     printLine(2, "KP:" + String(kp_r,2) + "KI:" + String(
193         ki_r,2));
194     printLine(3, "KD:" + String(kd_r,3));
195     delay(INTERVALO_MUESTREO_MS);
196 }
197 }
198 }
199 }

```

Listing 5: Código completo del Arduino para el sistema de levitación neumática.

Referencias

- [1] K. Ogata, *Ingeniería de Control Moderna*, 5ta ed. Madrid, España: Pearson Educación, 2010.
- [2] Almacenes El Rey. *Sitio web oficial*. Disponible en: <https://almaceneselrey.com>. Consultado: 10 de noviembre de 2025.
- [3] Colono Construcción. *Sitio web oficial*. Disponible en: <https://www.colonoconstruccion.com>. Consultado: 10 de noviembre de 2025.
- [4] Polycenter. *Sitio web oficial*. Disponible en: <https://polycenter.cr/index.html>. Consultado: 10 de noviembre de 2025.
- [5] CR Cibernética. *Sitio web oficial*. Disponible en: <https://www.crcibernetica.com>. Consultado: 10 de noviembre de 2025.
- [6] Benewake, “TF-Luna Product Manual,” SJ-PM-TF-Luna-A03. Disponible en: https://files.seeedstudio.com/wiki/Grove-TF_Mini_LiDAR/res/SJ-PM-TF-Luna-A03-Product-Manual.pdf. Consultado: 16 noviembre 2025.
- [7] MathWorks, “Designing Controllers with the PID Tuner,” *MATLAB & Simulink Documentation*. Disponible en: <https://www.mathworks.com/help/slcontrol/ug/designing-controllers-with-the-pid-tuner.html>. Consultado: 16 noviembre 2025.
- [8] T. Tkáčik, S. Jadlovská, M. Jadlovský, and M. Raso, “Design of Aerodynamic Ball Levitation Laboratory Plant,” *Processes*, vol. 9, no. 11, Art. 1950, 2021. Disponible en: <https://www.mdpi.com/2227-9717/9/11/1950>.